

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25%

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:25

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of

convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is considering **AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

Q5. Regularized CNN vs Non-Regularized CNN

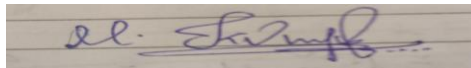
An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization and a CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

Code of Conduct:

I KUMARAN.M(192525022) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies

Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

ANSWERS

Q1. CNN vs Fully Connected Neural Network

A Convolutional Neural Network (CNN) is a deep learning architecture specifically designed for processing structured grid data such as images. A traditional Fully Connected Neural Network (FCNN) connects every neuron in one layer to every neuron in the next layer. For medical X-ray image classification, CNNs are generally more suitable because they preserve and exploit the spatial relationships between pixels.

1. Architectural design

A typical CNN consists of:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected → Output

The convolutional layers extract features, pooling reduces the spatial dimensions, and fully connected layers perform the final classification.

In an FCNN, the image is normally flattened into a one-dimensional vector before being given to the network.

For example, a (224×224) grayscale image contains:

[
224×224=50,176
]

pixel values. Flattening removes the natural 2D arrangement of these pixels.

2. Motivation

CNNs were developed to efficiently process images by exploiting:

- Local connectivity

- Parameter sharing
- Spatial relationships
- Hierarchical feature learning

FCNNs do not specifically exploit these properties.

3. Spatial feature extraction

CNNs preserve spatial information during convolution. A filter examines a small local region of the image and detects features such as:

- Edges
- Lines
- Corners
- Textures
- Shapes
- Abnormal regions

As the network becomes deeper, simple features are combined into more complex medical patterns.

4. Number of parameters

CNNs have considerably fewer parameters because the same filter weights are reused at different image locations.

For a convolutional layer:

$$[\text{Parameters} = (K_h \times K_w \times C_{in} + 1) \times C_{out}]$$

This makes CNNs much more parameter-efficient.

In contrast, a fully connected layer requires:

$$[\text{Parameters} = (N_{in} + 1) \times N_{out}]$$

For high-resolution images, this can become extremely large.

5. Computational complexity

Although CNNs perform many convolution operations, their local connectivity and parameter sharing make them much more efficient for image processing than connecting every pixel to every neuron. FCNNs become computationally expensive when the input image has a large number of pixels.

6. Role of convolutional layers

Convolutional layers apply filters to the image.

For example:

- First layer → edges
- Second layer → textures and simple shapes
- Middle layers → larger structures
- Deep layers → complex patterns and disease-related features

7. Role of filters

Filters are small learnable matrices such as:

[
3\times3,\quad 5\times5
]

They move across the image and generate feature maps.

Different filters learn different patterns.

8. Role of pooling

Pooling reduces feature-map dimensions.

For example, a (2\times2) max-pooling operation can reduce the spatial dimensions by approximately half.

Advantages:

- Reduces computation.
- Reduces memory usage.
- Retains important features.
- Provides some robustness to small shifts.
- Helps reduce overfitting.

9. Image-data handling

CNNs can accept images in their original spatial form:

[
Height\times Width\times Channels
]

An FCNN generally requires flattening the image, causing loss of explicit spatial structure.

10. Suitability for X-ray classification

X-ray images contain spatial patterns that are important for diagnosis. CNNs can automatically learn these patterns without requiring manual feature extraction.

For example, a CNN can learn features related to abnormal textures or regions that help distinguish between different classes.

Conclusion

CNNs are more suitable for X-ray classification because they preserve spatial information, automatically learn hierarchical features, use parameter sharing, require fewer parameters and provide efficient image processing. Therefore, CNNs are preferred over traditional fully connected networks for medical image classification.

Q2. Different CNN Layers and Filters

A CNN-based industrial defect detection system processes an image through several layers. Each layer performs a specific task, and together they transform raw pixels into meaningful features for classification.

CNN workflow

Input Image → Convolution → Activation → Pooling → Deeper
Convolution → Activation → Pooling → Fully Connected →
Classification

1. Convolutional layer

The convolutional layer is the main feature-extraction component of a CNN.

A filter slides across the image and performs multiplication and addition operations.

Conceptually:

[
Feature\ Map=Input*Filter
]

where (*) represents convolution.

Convolutional layers can detect:

- Edges
- Lines
- Corners
- Textures
- Cracks
- Scratches
- Surface irregularities

Example

For industrial products, an early convolutional layer might identify a sharp edge, while deeper layers can combine multiple edges to recognize a crack.

2. Filters

Filters are learnable kernels.

Common sizes include:

[
3×3, 5×5
]

Each filter learns a particular type of feature during training.

If a layer contains 32 filters, it can produce 32 feature maps, assuming the other dimensional details are appropriate.

3. Activation layer

The most commonly used activation function in CNNs is ReLU:

[
 $f(x) = \max(0, x)$
]

ReLU converts negative values to zero while retaining positive values.

Importance

It:

- Introduces non-linearity.
- Helps the network learn complex patterns.
- Makes deep networks more effective.
- Usually provides efficient training.

4. Pooling layer

Pooling reduces the spatial size of feature maps.

The common types are:

- Max pooling
- Average pooling

For example, max pooling with a (2×2) window selects the largest value from each region.

Advantages

- Reduces computation.
- Reduces memory requirements.
- Retains important information.
- Provides some translation robustness.

5. Fully connected layer

After feature extraction, the feature maps are converted into a suitable representation and passed to fully connected layers.

These layers combine the extracted features and make the final decision.

For example:

```
[  
[0.05,\ 0.92,\ 0.03]  
]
```

could represent probabilities for three classes, depending on the classification setup.

6. Early-layer filters

Early layers learn low-level features.

Examples:

- Edges
- Lines
- Corners

- Simple textures

These features are relatively general and can be useful for many types of images.

7. Deep-layer filters

Deeper layers combine low-level features into more complex patterns.

Examples:

- Complex shapes
- Surface structures
- Crack patterns
- Defect regions
- Object parts

Therefore:

Low-level features → Intermediate features → High-level features

Comparison

Feature	Early Layers	Deep Layers
Feature type	Simple	Complex
Examples	Edges, lines	Defect patterns, shapes
Spatial resolution	Usually higher	Usually lower
Semantic information	Low	High
Purpose	Basic feature extraction	High-level recognition

Conclusion

A CNN detects industrial defects through hierarchical feature extraction. Convolutional layers extract features, filters identify patterns, activation layers introduce non-linearity, pooling reduces dimensions, and fully connected layers perform classification. Early

filters detect simple patterns while deeper filters identify complex defects.

Q3. Parameter Sharing vs Independent Weights

Parameter sharing means that the same convolutional filter is reused at different locations of an image. This is fundamentally different from a fully connected network, where different connections generally have independent weights.

1. Parameter sharing in CNN

Suppose a (3×3) filter is used on an image.

Instead of learning separate weights for every position, CNNs use the same nine filter weights across the image.

For example:

Position 1 $\rightarrow$ same filter

Position 2 $\rightarrow$ same filter

Position 3 $\rightarrow$ same filter

and so on.

This dramatically reduces the number of trainable parameters.

2. Independent weights in FCNN

In a fully connected network, every connection has an independent weight.

If there are (N) input neurons and (M) output neurons:

[
Parameters= $N \times M$
]
plus biases.

For large images, (N) can be extremely large.

3. Parameter comparison

Consider a simplified image with 10,000 input values and a layer containing 1,000 neurons.

A fully connected layer would require approximately:

[
 $10,000 \times 1,000 = 10,000,000$
]

weights, excluding biases.

A convolutional layer may use only a relatively small number of filter weights that are reused across the image.

This demonstrates the parameter efficiency of CNNs.

4. Memory requirements

Fewer parameters mean:

- Lower memory consumption.
- Smaller model size.
- Easier storage.
- More efficient deployment.

This is especially important for large-scale surveillance systems processing thousands of images.

5. Translation-related robustness

Suppose an object appears on the left side of one image and the right side of another.

Because CNN filters are applied throughout the image, the same filter can detect the object-related feature in both locations.

This provides translation-related robustness.

It is important to note that parameter sharing alone does not provide complete translation invariance; pooling and other architectural choices contribute to robustness as well.

6. Learning efficiency

Parameter sharing means that the network does not have to learn separate detectors for the same type of feature at every location.

For example, an edge detector can be useful whether the edge occurs at the top, bottom, left or right of an image.

Advantages

1. Fewer parameters.
2. Lower memory usage.
3. Reduced computational requirements.
4. Faster learning.
5. Better reuse of learned features.
6. Improved ability to detect features at different locations.
7. Suitable for large-scale image processing.

Conclusion

Parameter sharing is a major reason CNNs are efficient for image processing. It allows the same filter to detect the same type of feature throughout an image, significantly reducing the number of parameters compared with independent weights in fully connected networks.

Q4. AlexNet vs ResNet

AlexNet and ResNet are both CNN architectures used for image classification. However, ResNet introduced residual connections that made it possible to train substantially deeper networks.

1. Network depth

AlexNet has a relatively shallow architecture compared with modern deep CNNs.

ResNet is available in several depths, such as:

- ResNet-18
- ResNet-34
- ResNet-50
- ResNet-101
- ResNet-152

Therefore, ResNet can learn more complex hierarchical representations.

2. Layer organization

AlexNet

A simplified structure is:

Input → Convolution → ReLU → Pooling → Convolution → ReLU
→ Pooling → Fully Connected → Output

ResNet

ResNet uses residual blocks with shortcut connections.

Conceptually:

[

Output=F(x)+x

]

where:

- (x) = input
- $(F(x))$ = learned residual mapping

3. Convolutional filters

AlexNet famously used relatively large filters in its early layers, including an (11×11) filter in the original architecture.

ResNet architectures commonly rely heavily on (3×3) convolutions, with bottleneck blocks in deeper variants such as ResNet-50.

4. Feature extraction

AlexNet can learn:

- Edges
- Textures
- Shapes
- Object patterns

ResNet can learn deeper hierarchical representations, making it particularly effective for complex recognition tasks.

5. Parameter requirements

AlexNet contains a large number of parameters, with a substantial portion in its fully connected layers.

ResNet's parameter count depends on the version. Deeper ResNets can still be computationally demanding, but residual blocks make deep feature extraction practical.

6. Training difficulty

As networks become deeper, training becomes difficult because of optimization problems such as vanishing or degraded gradients.

ResNet addresses this using shortcut connections.

7. Vanishing-gradient problem

During backpropagation, gradients may become very small as they pass through many layers.

ResNet provides shortcut paths through which gradients can flow more directly.

Therefore, deeper ResNet models can be trained more effectively.

8. Computational cost

AlexNet is generally less computationally demanding than very deep ResNet variants.

ResNet-50 and deeper versions require more computation, but they generally provide stronger feature extraction and classification performance.

9. Classification performance

For modern complex image-recognition applications, ResNet is generally preferred because of:

- Deeper architecture.
- Residual learning.
- Better optimization of deep networks.
- Strong hierarchical feature extraction.
- Strong classification performance.

Final comparison

Feature	AlexNet	ResNet
Depth	Relatively shallow	Can be very deep
Architecture	Sequential	Residual blocks

Feature	AlexNet	ResNet
Skip connections	No	Yes
Early filters	Large filters in original design	Primarily smaller convolutions
Deep feature learning	Good	Excellent
Training very deep networks	Difficult	Easier
Vanishing/degradation issues	More susceptible	Reduced
Computational cost	Lower	Higher for deeper versions
Complex recognition	Good	Generally better

Recommendation

For a complex image-recognition system, I recommend ResNet, particularly an appropriate ResNet variant selected according to available computational resources.

Conclusion: ResNet is more suitable when the objective is strong performance on complex image-recognition tasks, while AlexNet may be useful when a simpler and older architecture with lower computational demands is preferred.

Q5. Regularized CNN vs Non-Regularized CNN

In the given agricultural application, the CNN achieves very high training accuracy but performs poorly on unseen images. This is a classic indication of overfitting.

Overfitting occurs when a model learns the training data too specifically and fails to generalize to new data.

1. CNN without regularization

A non-regularized CNN may:

- Memorize training examples.
- Achieve very high training accuracy.
- Perform poorly on validation/test data.
- Learn irrelevant details such as background patterns.
- Generalize poorly.

For example:

Training accuracy = 98%

but

Validation accuracy = 75%

would indicate a significant generalization problem.

2. Regularized CNN

A regularized CNN introduces techniques that reduce overfitting and improve generalization.

Important techniques include:

A. Dropout

Dropout randomly sets some neuron outputs to zero during training.

For example, with a dropout rate of 0.5, approximately half of the selected activations are dropped during a training step.

This prevents the network from depending too heavily on particular neurons.

Advantages:

- Reduces overfitting.
- Encourages robust feature learning.
- Improves generalization.

B. Data augmentation

Data augmentation generates varied training examples from existing images.

Useful transformations for leaf images include:

- Rotation
- Horizontal/vertical flipping where biologically appropriate
- Cropping
- Scaling
- Translation
- Moderate brightness variation

For example, one leaf image can produce several slightly different training examples.

This increases data diversity and helps the model recognize diseases under different real-world conditions.

C. Batch normalization

Batch normalization normalizes activations within training batches.

It can:

- Improve training stability.
- Support faster convergence.
- Make optimization easier.

- Provide some regularizing effect.

It is commonly placed after convolution and before or around the activation depending on the architecture.

3. Effect on overfitting

Method	Effect on Overfitting
No regularization	High risk
Dropout	Reduces over-dependence on individual neurons
Data augmentation	Increases data diversity
Batch normalization	Improves training stability and may provide regularization

4. Generalization

Generalization means the ability of the CNN to perform well on images that were not used during training.

A regularized model generally has better generalization because it learns meaningful disease-related features instead of memorizing the training examples.

5. Training stability

Batch normalization can make optimization more stable.

Dropout introduces randomness during training, which also helps prevent over-dependence on particular paths through the network.

Data augmentation exposes the model to greater variation.

6. Computational requirements

Regularization can introduce some additional computation during training.

However, the improvement in generalization is usually more important for a real-world application than minimizing training cost.

Data augmentation may increase the number of training examples, while dropout and batch normalization add operations during training.

7. Recommended approach

For plant disease classification, the following approach is appropriate:
Input leaf image → Data augmentation → CNN convolutional layers → Batch normalization/activation → Pooling → Dropout where appropriate → Classification

The exact placement and amount of dropout should be selected using validation performance rather than applying an unnecessarily high dropout rate.

8. Important practical consideration

Regularization should not be excessive.

Too much dropout or excessive augmentation can cause underfitting, where the model fails to learn useful patterns.

Therefore, regularization should be balanced using a validation dataset.

Final recommendation

The regularized CNN is more suitable for reliable real-world plant disease classification. Data augmentation should be used to increase image diversity, batch normalization to improve training stability, and dropout where needed to reduce overfitting.

Conclusion

A non-regularized CNN may achieve excellent training accuracy but poor performance on unseen images because of overfitting. A properly regularized CNN provides better generalization, improved robustness

and more reliable performance on real-world plant images. Therefore, a combination of data augmentation, batch normalization and suitable dropout is recommended.